CLOUD NATIVE
COMPUTING FOUNDATION

Slurm-on-K8s For DDP

Adaptive HPC Scheduling on
Cloud Native Infrastructure
For Distributed Training

2025.04.30

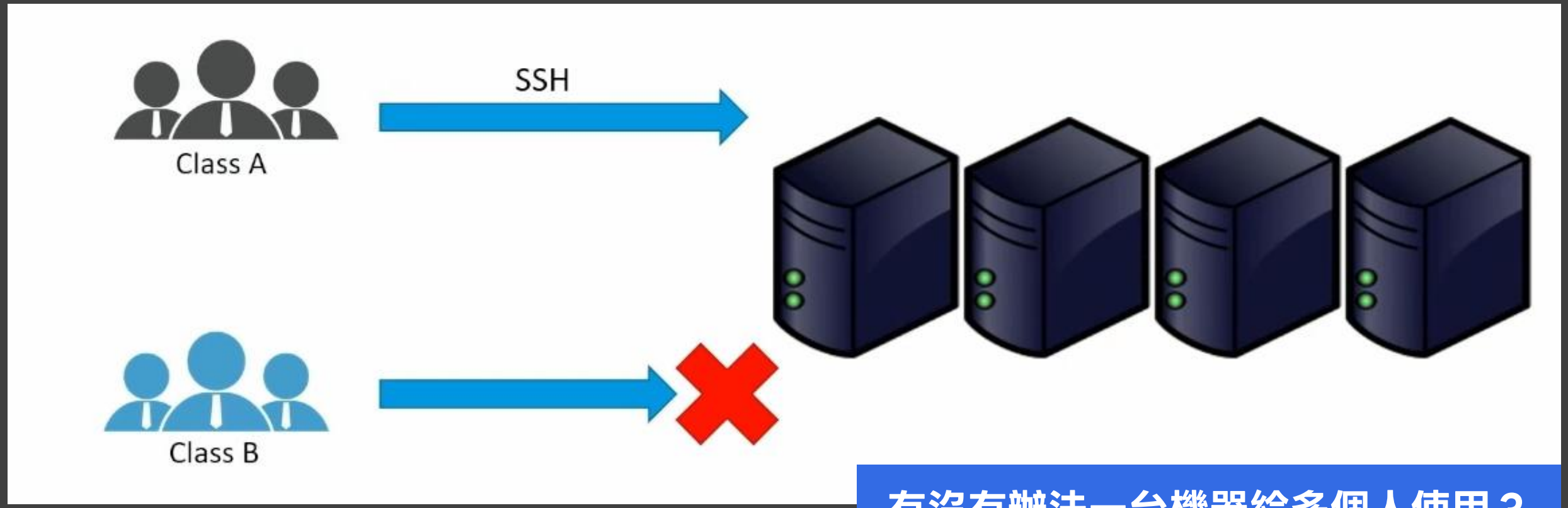
Agenda

1. Introduction
2. System Architecture
3. Demo & Evaluation
4. References

01. Introduction

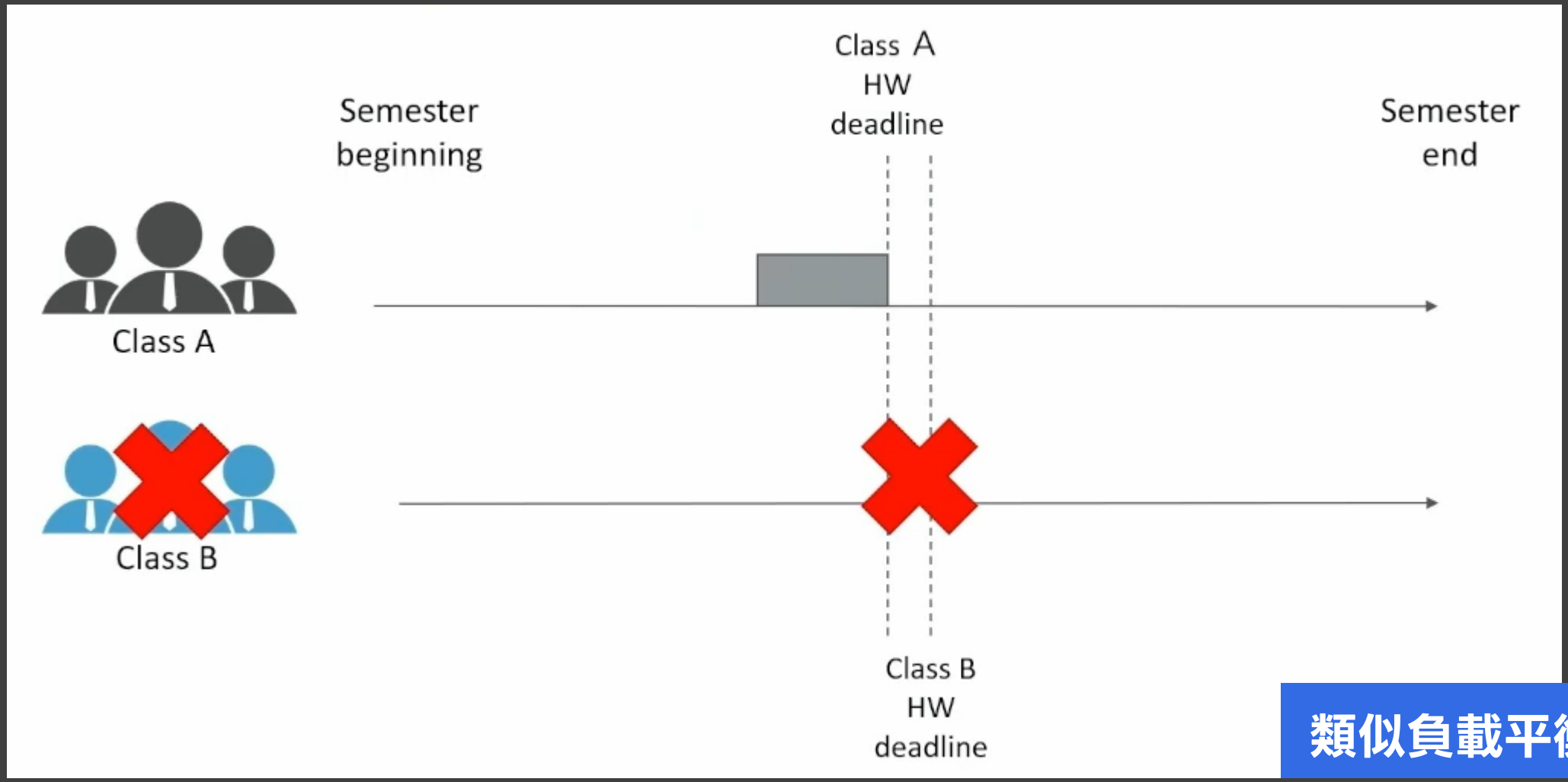
專題簡介

Example Scenarios



有沒有辦法一台機器給多個人使用？

Example Scenarios



類似負載平衡的概念

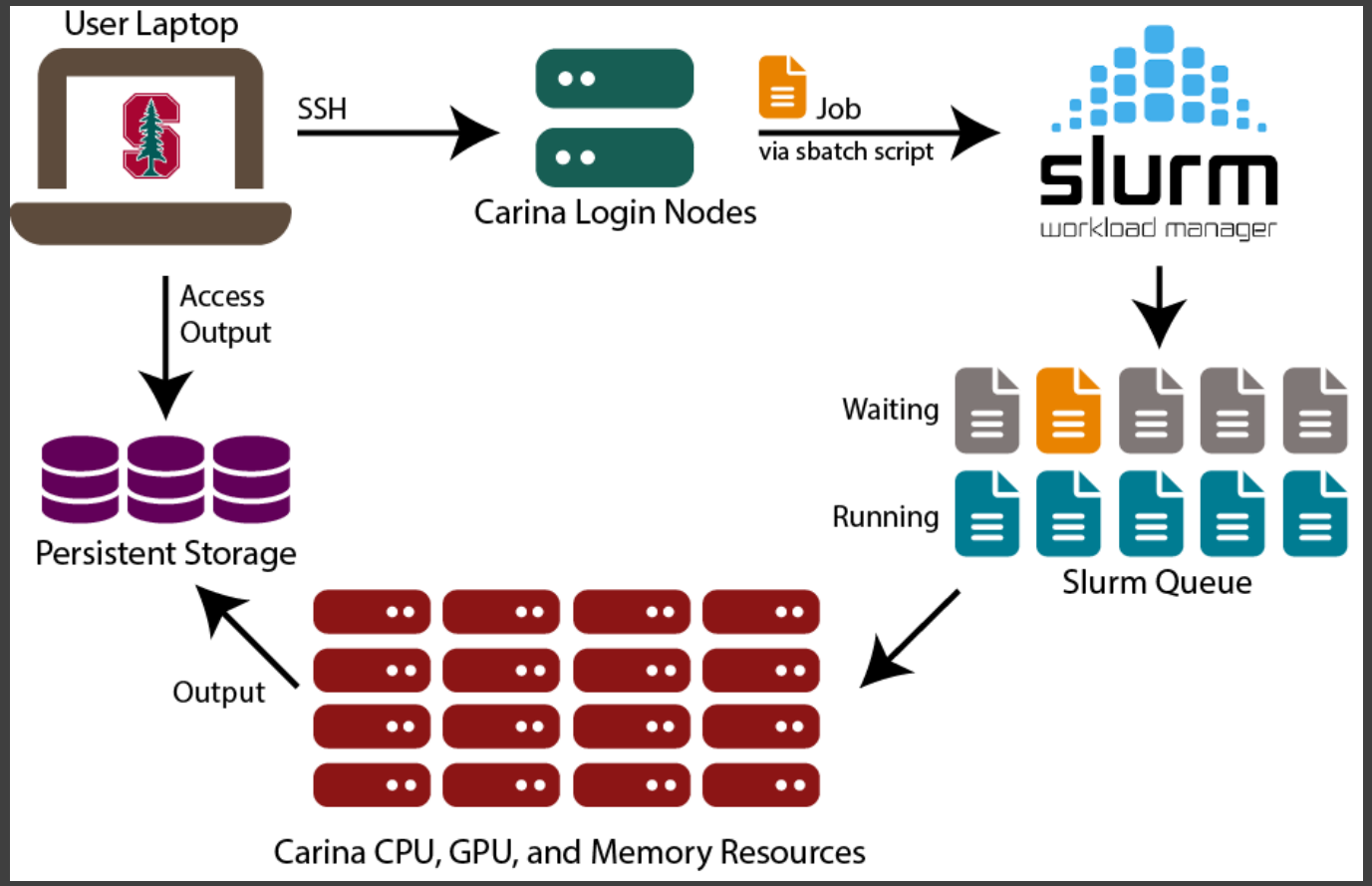
Slurm

What is Slurm？

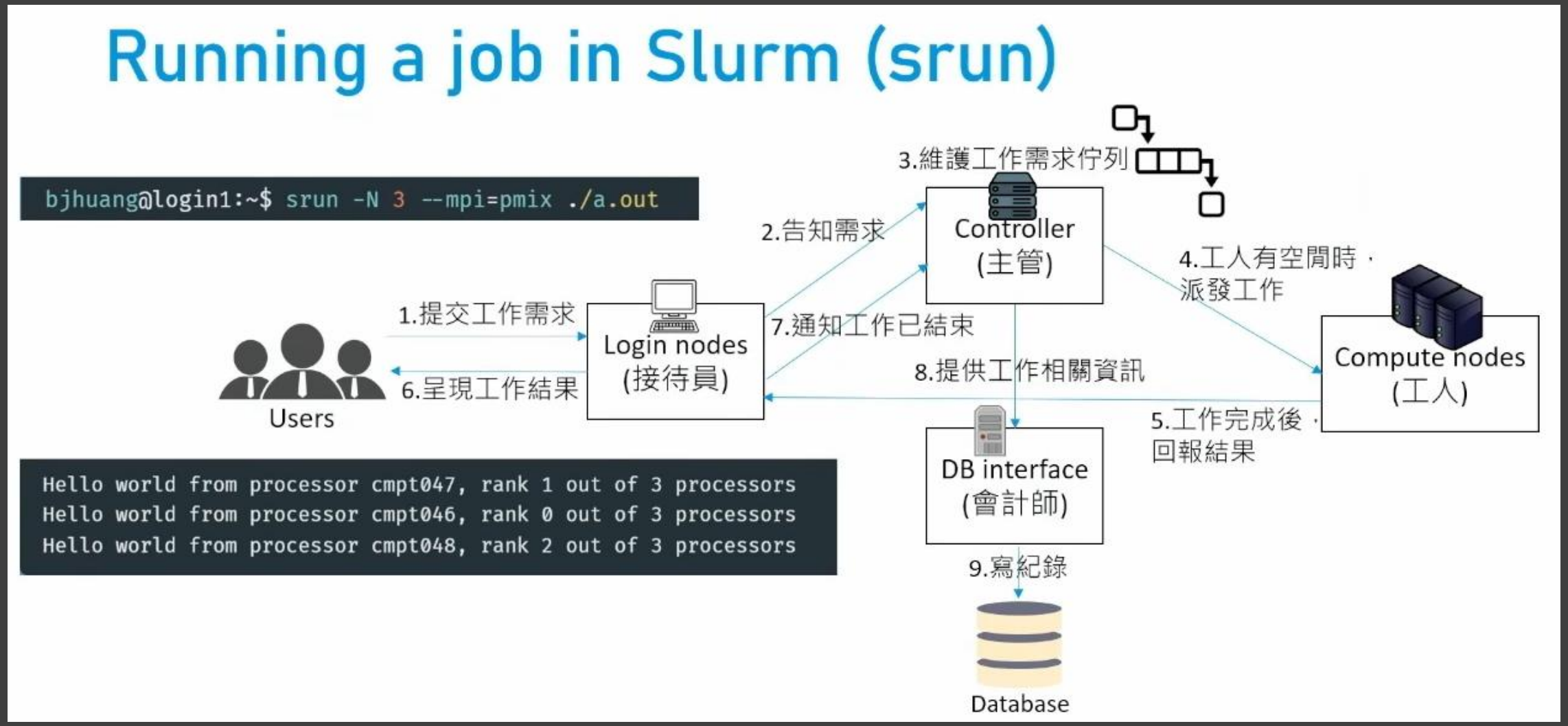
- Slurm = Simple Linux Utility for Resource Management
- An open source system
  - 管理工作：資料分配、排程、執行

Roles in Slurm

- Login nodes = 接待員/客服
- Controller = 主管
- Compute nodes = 工人
- DB interface = 會計師



Running a job in Slurm (srun)



Kubernetes

What is Kubernetes？

- Kubernetes (K8s) = Containers Orchestration Tool
- An open source system
  - 管理容器：啟動、運算資源、排程、自動部署
  - 大公司架設服務很愛用

Background

Motivation

- Kubernetes 很擅長資源編排與彈性伸縮，但對 HPC / AI 訓練工作負載的語意支援有限。
- Slurm 很擅長批次排程、資源配額與叢集治理，但傳統部署通常偏靜態，對雲端環境的節點生命週期不夠友善。

⟹ 結合兩者來做 AI/ML 分散式訓練

Benefits

- 提升資源利用率
  - Slurm 可以透過將工作排程到 K8s 叢集中的可用資源
- 簡化的工作負載管理
  - 一律透過 Slurm 介面，用於管理 ML 工作負載
- 提升可擴展性
  - Slurm 可以透過將機器學習工作負載排程到多個 K8s 叢集來擴展

Present Solutions

- Slurm-on-Kubernetes：CoreWeave’s SUNK
- Slinky 專案
  - slurm-operator：用 K8s CRD 管理 Slurm 的 controller / login / nodeset（把 Pod 當成 Slurm node）。
  - slurm-bridge：讓 Slurm 可以把 workload 映射到 K8s Pod。

Slurm on Kubernetes

- Slurm 決定誰用資源、什麼時候用、用多少
  - 計算資源排程問題
- Kubernetes 決定資源怎麼存在、怎麼啟動、怎麼管理
  - 動態提供 Slurm 需要的運算節點
  - 當硬體調整的時候不用重新調整 Slurm 設定
  - 依賴版本管理、網路設定

Our Feature !

- 擴容、擴縮、低功耗、節點壞掉重啟
- 客製化排程演算法、運算資源分配
- 針對分散式訓練 (DDP)
- 近年來企業很重視安全，可以裝個龍蝦
- Checkpoint Guard

Reliability

Performance

Lifecycle Unity

Observability

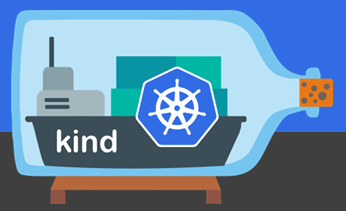
Target Audience (TA)

- 學術單位、企業 AI 平台，多個研究小組共用 GPU 叢集
- AI 研究平台工程師（學術單位、企業 MLOps）
  - 一鍵部署、多租戶隔離、SLO 告警整合到既有 on-call 流程。
- 雲端架構師 / SRE：需要 HA、可觀測性、Terraform / ArgoCD 整合的 IaC 交付形式。

02. System Architecture

系統設計與架構

Development Environment



- Docker Desktop + Kind (Kubernetes in Docker)
- Kind cluster (slurm-lab)　單節點本機叢集，模擬多 Pod 場景
- Controller&Worker：Ubuntu 22.04 + Slurm + Munge + OpenSSH
- Operator：Python 3.11-slim + kubectl CLI

https://kind.sigs.k8s.io/

User Scenarios (1/2)

- **情境一：提交 CPU 批次訓練任務**
- 使用者透過 Login Pod 執行 sbatch，Operator 偵測到 pending jobs 後自動擴容 CPU 工作節點

- **情境二：提交 GPU DDP 分散式訓練**
- 使用 --constraint=gpu-a10 或 --gres=gpu:a10:1，Operator 自動擴容對應 GPU 節點池

User Scenarios (2/2)

- **情境三：任務完成後自動縮容**
- Queue 清空後，Operator 等待 cooldown（預設 60s），將閒置節點縮回 baseline（CPU=1, GPU=0）

- **情境四：Checkpoint 保護機制**
- 任務執行中若 checkpoint 過時（超過 10 分鐘），Operator 阻止縮容，避免訓練狀態遺失

System Design

- **無 CRD 框架的輕量 Operator**
- 純 Python + kubectl CLI，不依賴 Kopf 或自訂 CRD，降低部署複雜度
- **多節點池獨立擴縮**
- CPU / GPU-A10 / GPU-H100 三個節點池，各自獨立的 min/max replicas、scale step、cooldown 策略
- **Checkpoint-Aware 縮容保護**
- 縮容前檢查 checkpoint 檔案時間戳，若過舊則阻止，保障 DDP 訓練容錯能力

Slurm GRES (1/3)

- Slurm 的通用資源管理機制，讓 Slurm 能夠追蹤和排程 CPU/Memory 以外的資源 (e.g. GPU)
- 知道每個 node 上有哪些 GPU（型號、數量）
- 讓使用者提交 job 時指定需要的 GPU 類型和數量
- 確保 GPU 資源不會被過度分配

Slurm GRES (2/3)

- Slurm 可以自動偵測，但在 Kind 模擬要手動設定 vGPU

DDP Workflow

```
DDP on Slurm 的運作原理

sbatch -N 2 train.sbatch
    ↓
Slurm 分配 slurm-worker-cpu-0 + slurm-worker-cpu-1
    ↓
每個節點跑 srun → 在每個節點上 launch torchrun
    ↓
torchrun 啟動 N 個 process（每個 GPU 一個）
    ↓
process 透過 NCCL/Gloo 做 AllReduce（梯度同步）
    ↓
checkpoint 寫到 /shared（所有節點都看得到）

關鍵是：Slurm 提供 SLURM_NODELIST、SLURM_PROCID、SLURM_NNODES 這些環境變數，torchrun 需要從這裡知道 MASTER_ADDR 是哪個節點。
```

Observability

Observability

雲端服務崛起

錯誤排查難度上升

Observability

Slurm	Bridge	K8s
Queue depth	Operator scale decision	StatefulSet replicas
Job wait-time	Scale-up event	Pod ready time
Node idle-time	Scale-down decision	Replica count drop
Checkpoint age	Guard blocked / allowed	Scale event skipped

Prometheus

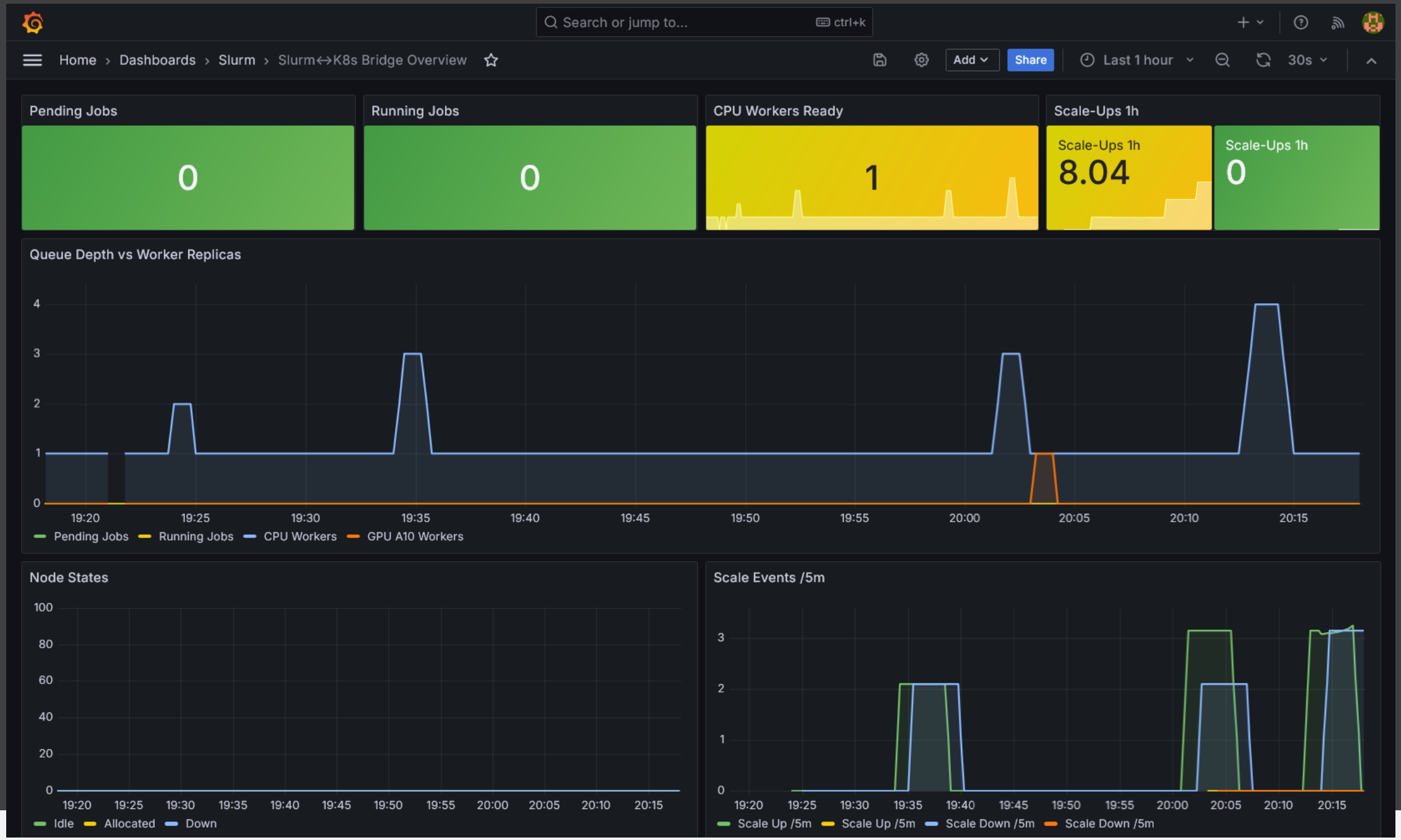
- 最受歡迎的開源監控與告警工具組，是 CNCF 的畢業專案
- 專為動態雲端環境設計，擅長透過 HTTP 拉取模型收集指標，儲存於時間序列資料庫中，並使用 PromQL 進行數據查詢分析。
- 負責發 Alert 到 Slack/Telegram

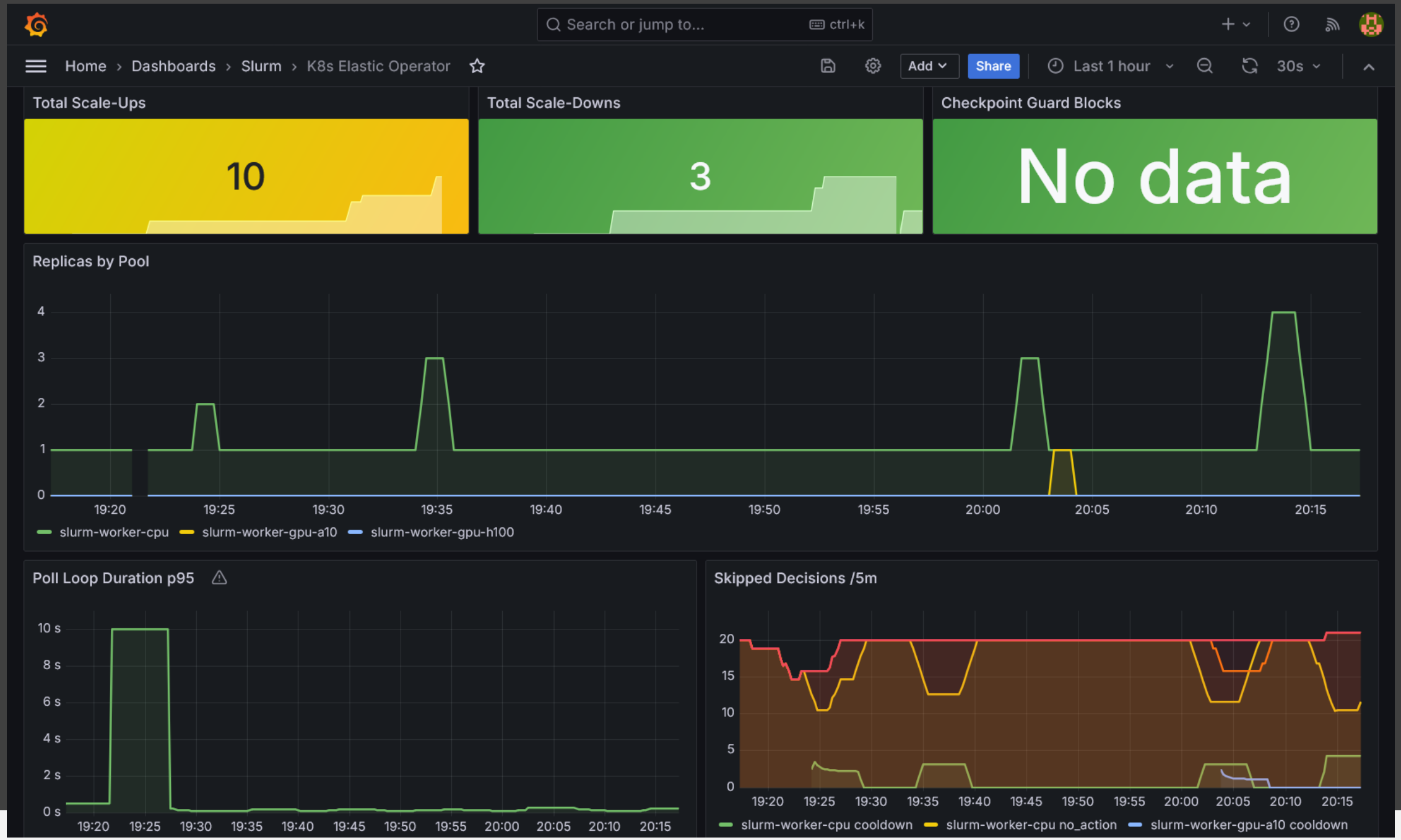
https://wade-software-study-note.medium.com/devops%E8%AA%B2%E7%A8%8B-prometheus-1-7a690f7d4426

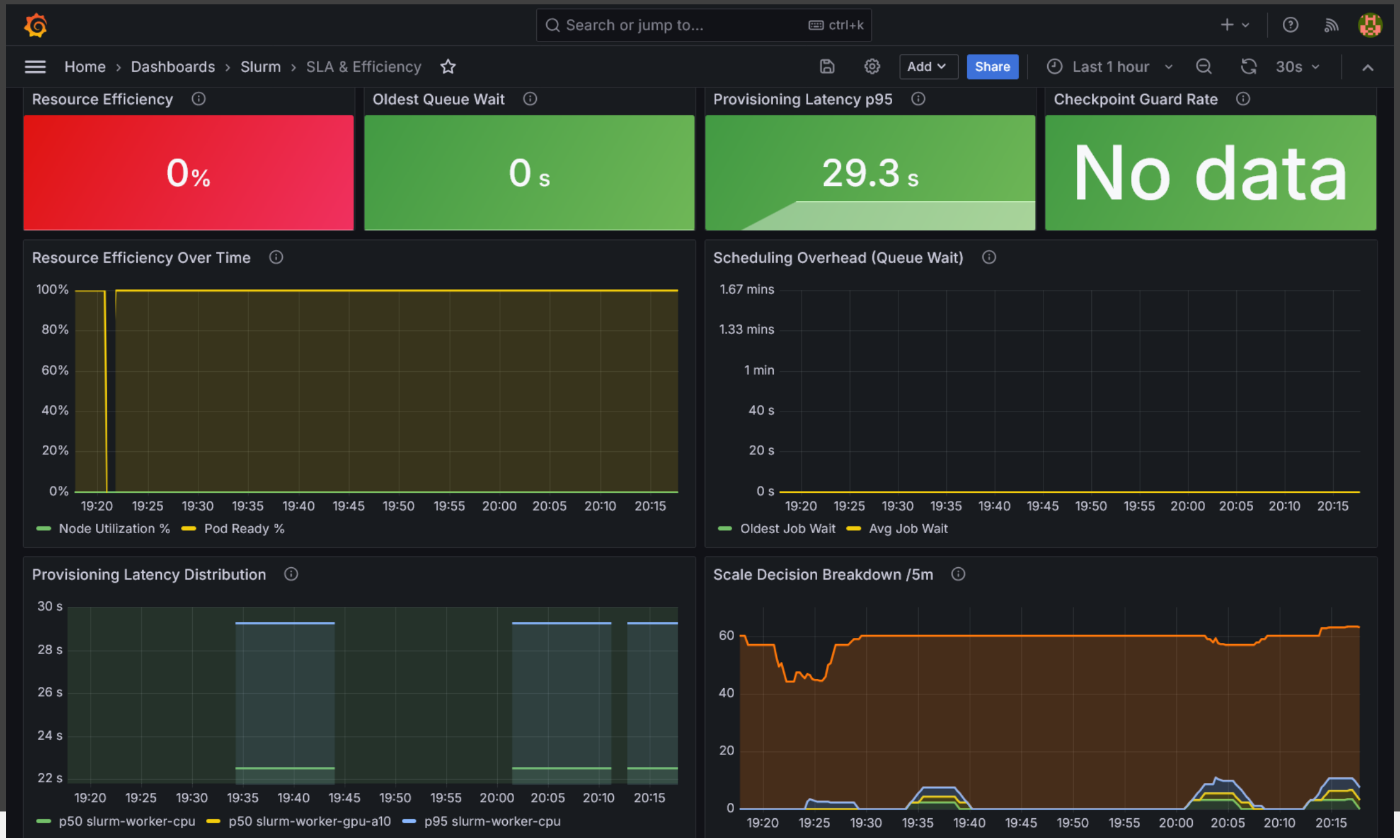
Grafana

- 能夠將各種資料做成監控儀表板，支援多種資料來源，以及提供豐富的面板選擇，協助我們將資料視覺化。
- Prometheus 預設的 GUI 功能不足，需要 Grafana 這種支援豐富圖表的工具幫助我們可視化 Metrics 資訊。

https://www.figma.com/deck/O8BwdzmRUfedbq0s8w7gm8/sitcon2026-grafana-v3?node-id=21-158

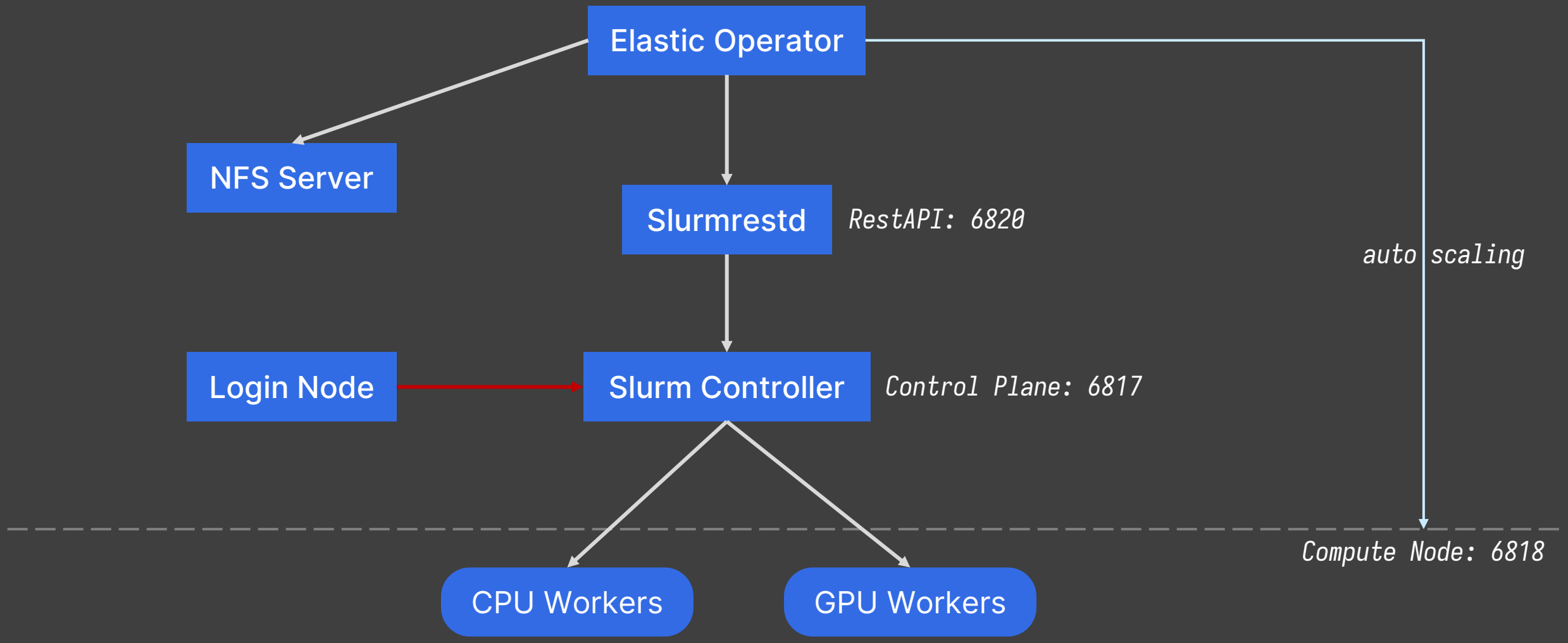

Search or jump to...
ctrl+k
Home › Dashboards › Slurm › Slurm↔K8s Bridge Overview
Add
Share
Last 1 hour
30s
Pending Jobs
0
Running Jobs
0
CPU Workers Ready
1
Scale-Ups 1h
Scale-Ups 1h
8.04
Scale-Ups 1h
0
Queue Depth vs Worker Replicas
4
3
2
1
0
19:20
19:25
19:30
19:35
19:40
19:45
19:50
19:55
20:00
20:05
20:10
20:15
Pending Jobs
Running Jobs
CPU Workers
GPU A10 Workers
Node States
100
80
60
40
20
0
19:20
19:25
19:30
19:35
19:40
19:45
19:50
19:55
20:00
20:05
20:10
20:15
Idle
Allocated
Down
Scale Events /5m
3
2
1
0
19:20
19:25
19:30
19:35
19:40
19:45
19:50
19:55
20:00
20:05
20:10
20:15
Scale Up /5m
Scale Up /5m
Scale Down /5m
Scale Down /5m


Search or jump to...
ctrl+k
Home › Dashboards › Slurm › K8s Elastic Operator
Add
Share
Last 1 hour
30s
Total Scale-Ups
10
Total Scale-Downs
3
Checkpoint Guard Blocks
No data
Replicas by Pool
4
3
2
1
0
19:20
19:25
19:30
19:35
19:40
19:45
19:50
19:55
20:00
20:05
20:10
20:15
slurm-worker-cpu
slurm-worker-gpu-a10
slurm-worker-gpu-h100
Poll Loop Duration p95
10 s
8 s
6 s
4 s
2 s
0 s
Skipped Decisions /5m
20
15
10
5
0
slurm-worker-cpu cooldown
slurm-worker-cpu no_action
slurm-worker-gpu-a10 cooldown


Search or jump to...
ctrl+k
Home › Dashboards › Slurm › SLA & Efficiency
Add
Share
Last 1 hour
30s
Resource Efficiency
0%
Oldest Queue Wait
0 s
Provisioning Latency p95
29.3 s
Checkpoint Guard Rate
No data
Resource Efficiency Over Time
100%
80%
60%
40%
20%
0%
19:20 19:25 19:30 19:35 19:40 19:45 19:50 19:55 20:00 20:05 20:10 20:15
Node Utilization %
Pod Ready %
Scheduling Overhead (Queue Wait)
1.67 mins
1.33 mins
1 min
40 s
20 s
0 s
19:20 19:25 19:30 19:35 19:40 19:45 19:50 19:55 20:00 20:05 20:10 20:15
Oldest Job Wait
Avg Job Wait
Provisioning Latency Distribution
30 s
28 s
26 s
24 s
22 s
19:20 19:25 19:30 19:35 19:40 19:45 19:50 19:55 20:00 20:05 20:10 20:15
p50 slurm-worker-cpu
p50 slurm-worker-gpu-a10
p95 slurm-worker-cpu
Scale Decision Breakdown /5m
60
40
20
0
19:20 19:25 19:30 19:35 19:40 19:45 19:50 19:55 20:00 20:05 20:10 20:15

Networking

Network Flow



https://github.com/SoWiEee/Slurm-on-K8s-For-DDP/blob/main/docs/cluster.md

Future Work

Some Improvements

- 目前有 2 張網卡切成 2 個獨立子網
- 目前 bridge CNI 沒有 QoS，兩個介面走同一個 veth，沒有隔離
- 真實 HPC 用 InfiniBand 或 RoCE v2 達到 400 Gbps 低延遲互連
  - 容器內部 veth 的延遲約 $50 \sim 100\ \mu s$，IB 的 latency 約 1–2 μs，差距 50 倍以上

(+) Tools：Network Operator + SRIOV Device Plugin

https://github.com/Mellanox/network-operator | https://github.com/k8snetworkplumbingwg/sriov-network-device-plugin

Helm Chart

- 切分容器/服務，部署需要依序執行多支 bootstrap 腳本。Helm 讓整個系統可以一條指令完成：

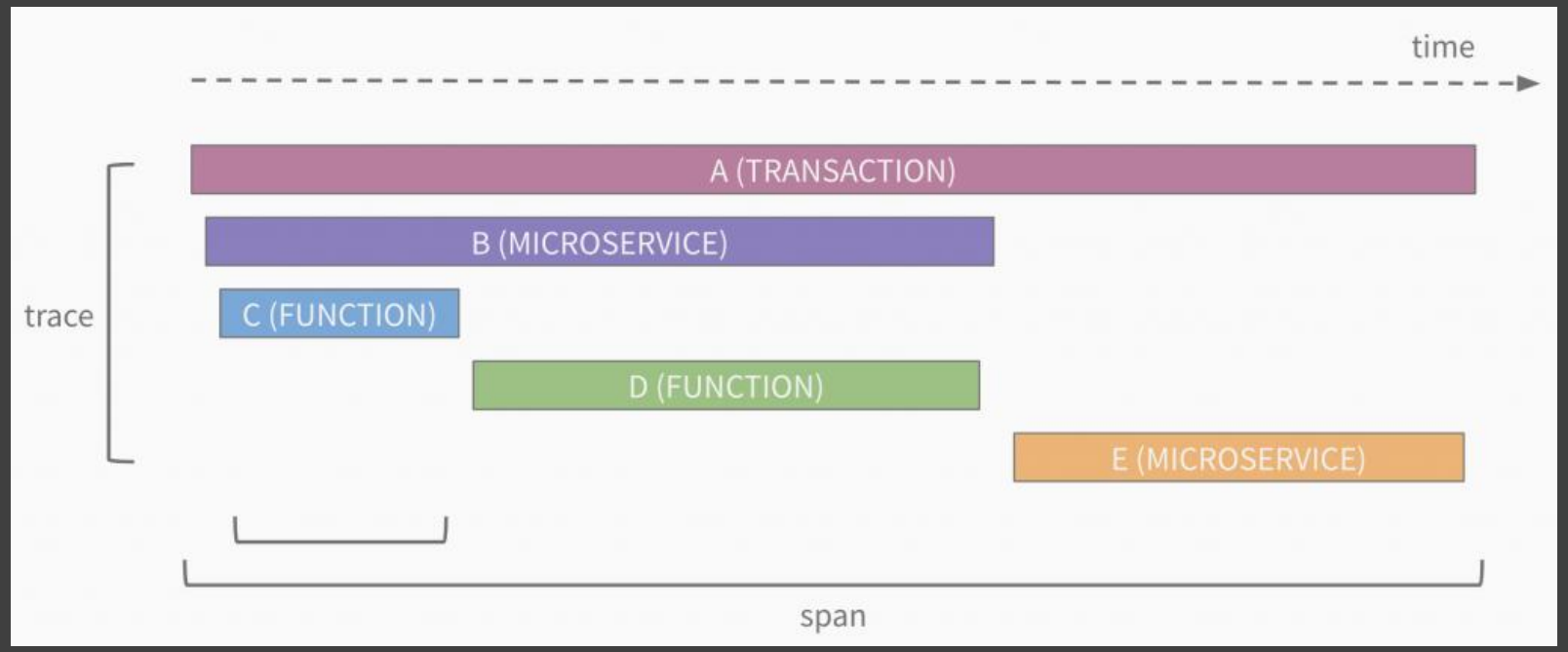
```
helm install slurm-on-k8s ./chart \
  --set cluster.name=my-lab \
  --set pools.cpu.maxReplicas=8 \
  --set pools.gpuA10.maxReplicas=4 \
  --set monitoring.enabled=true \
  --set alertmanager.slack.webhookUrl="https://hooks.slack.com/..."
```

- 主要收益：dev/prod 只需一份 yaml，消除目前「改完 JSON 還要重新 render manifest」的步驟。

https://helm.sh/docs/topics/charts/

Distributed Tracing (1/2)

- 搜集應用程式內部與端到端請求相關的資料，以實現對整個系統的監控和分析。



Distributed Tracing (2/2)

- 讓一個訓練 job 的完整生命週期變成一條可視化的 Trace：

```
[sbatch submit] → [pending in queue] → [scale-up decision]
  → [K8s provisioning] → [slurmd registration] → [DDP torchrun]
    → [checkpoint write] → [scale-down decision] → [job complete]
```

- 紀錄 job_id, pool, checkpoint_age 等屬性

(+) Tools：Jaeger / Grafana Tempo

https://grafana.com/oss/tempo/

High Availability (1/2)

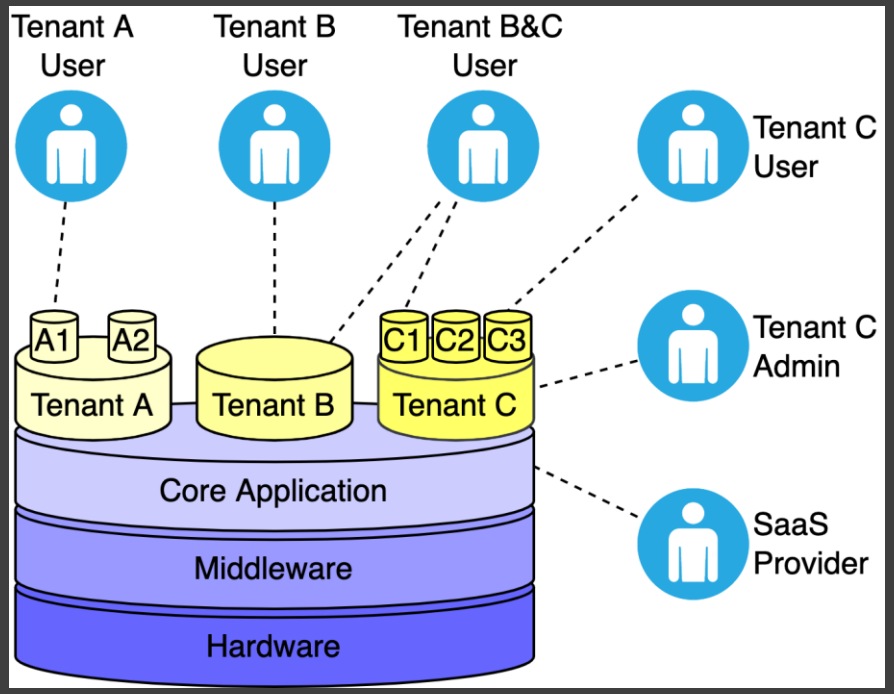
- 企業服務越來越依賴網路系統，若系統一當機就可能造成營收損失或商譽危機，因此備援與 HA 是資訊系統設計不可忽視的一環。
- 系統在遭遇故障、維護或負載高峰時，仍保持服務不中斷。
  - 減少單點故障 (SPOF)
  - 提供故障轉移 (failover) 能力
  - 達到接近 7x24 小時無間斷的服務水準

High Availability (2/2)

- 目前只有一個 Operator，Pod 重啟會有 15~30 秒的決策空窗。
- Leader Election via K8s Lease（coordination.k8s.io/v1）：多個 Operator Pod 同時運行，只有 leader 執行 scaling loop
- Cooldown 狀態已透過 StatefulSet annotation 持久化，重新選主不會重置 cooldown
- 🎯 Zero-downtime Operator 滾動升級

Fair-Share Multi-Tenancy (1/2)

- 多租戶技術是一種軟體架構，允許單一軟體同時服務多個租戶或客戶，而這些租戶之間的資料和操作是完全獨立的。
- 實現多個客戶之間的隔離性、安全性。
- 可以實現多個客戶共享相同的 infra 和服務，從而節省資源並提高效率。



https://www.cloudflare.com/learning/cloud/what-is-multitenancy/

Fair-Share Multi-Tenancy (2/2)

- 支援多個 team / project 共用同一個叢集
- 🎯 每個 team 有自己的 Slurm 帳戶和 shares 配額
- 🎯 需要創建 Slurm 會計系統 slurmdbd
- Operator 可依優先度調整各 pool 的 scale-up 順序

(+) Metrics：per-account queue depth、cumulative CPU-hours、FairShare score

https://slurm.schedmd.com/classic_fair_share.html

QoS (1/4)

- 所有節點在同一個 partition，MaxTime=INFINITE
- 真實 HPC 叢集通常有：

```
PartitionName=interactive Nodes=cpu-[0-3] MaxTime=4:00:00 Priority=50
PartitionName=gpu-normal  Nodes=gpu-a10-[0-3] MaxTime=24:00:00 Priority=100 QOS=gpu-normal
PartitionName=gpu-preempt Nodes=gpu-h100-[0-3] MaxTime=INFINITE PreemptMode=REQUEUE
```

- 沒有 MaxTime 限制，代表一個壞掉的 job 可以永遠占用節點

https://github.com/SoWiEee/Slurm-on-K8s-For-DDP/blob/main/docs/review.md

QoS (2/4)

- 排程策略：沒有 Preemption 設定
- 若高優先度的緊急工作負載（例如 model serving 或緊急訓練）提交，無法搶佔已佔用資源的低優先度 job。
- 改進作法：

```
PreemptType=preempt/partition_prio
PreemptMode=REQUEUE    # 被搶佔的 job 重新排隊
# 或
PreemptMode=CHECKPOINT  # 被搶佔前先存 checkpoint
```

QoS (3/4)

- 排程策略：沒有 Gang Scheduling
  - DDP Job 需要**所有 rank 同時運行**，而 Slurm 預設是只要申請的節點有空就開始跑，剩餘的 rank 等待。
  - 在大型叢集會造成 Backfill fragmentation：部分 rank 已佔用 GPU，整個 job 仍無法開始，浪費資源。
- 解決方案：
  - Gang Scheduling (KEP-4671) @k8s v1.35

QoS (4/4)

- Backfill scheduler 有幾個重要參數目前是 Default

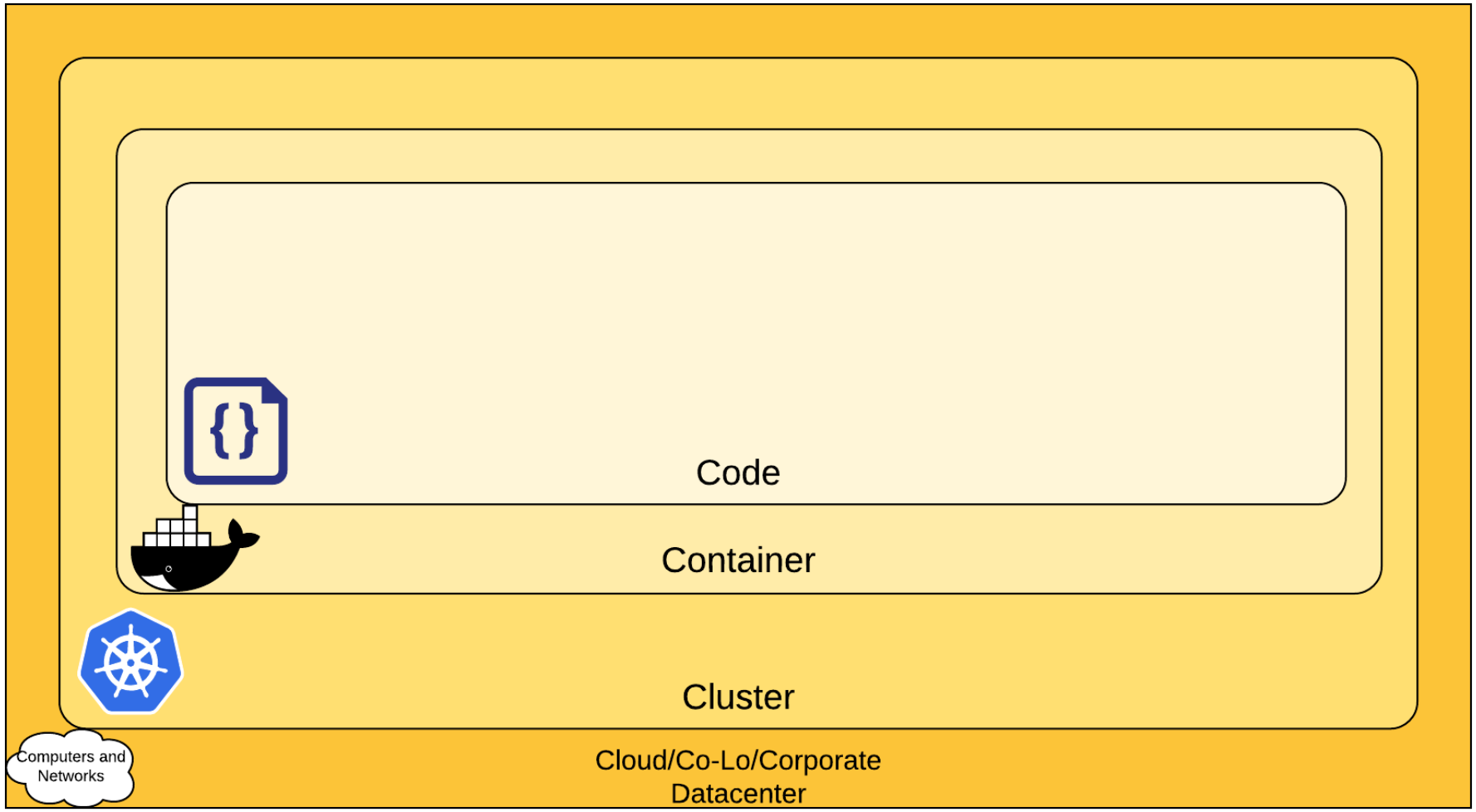
```
SchedulerType=sched/backfill
SchedulerParameters=bf_max_job_user=10,bf_max_job_test=500,bf_resolution=600
```

- 在 GPU 叢集中，若 backfill 時間解析度太大，scheduler 無法正確預測短時間內的 GPU 釋放，導致 backfill 效果差。

https://github.com/SoWiEee/Slurm-on-K8s-For-DDP/blob/main/docs/review.md

Security

The 4C's of Cloud Native Security



https://www.tigera.io/learn/guides/cloud-native-security/

Code Security

- 雖然應用程式原始碼的安全通常超出了 K8s 基礎設施的討論範圍，但它依然是整個雲端原生安全不可或缺的一環。
- 實施靜態/動態應用程式安全測試 (SAST / DAST)
- 定期掃描與更新第三方程式庫與相依套件
- 防範常見的應用程式漏洞 (OWASP Top 10)

Container Security

- 運行環境的權限設定，最小權限原則
- 容器內部軟體是否本身就有安全性漏洞
- Image 本身的數位簽章

Cluster Security

- 主要針對 K8s 叢集本身的防護
- 必須根據應用程式的攻擊面來配置安全策略：
  - RBAC/Auth
  - Secrets Management
  - Pod 安全規範
  - QoS & Resource limit
  - Network Policies
  - TLS 加密

Cloud Security

- 這是整個安全模型的地基，也是 Kubernetes 叢集的信任基礎
- 如果雲端層本身存在漏洞，或者設定不當，那麼建構在其上的所有元件都無法保證安全。
- 嚴格遵守各家雲端服務供應商（AWS, GCP, Azure）針對其環境所提出的基礎設施安全建議與最佳實踐。

03.
Demo/Evaluation

演示與評估

04. References

參考資料與文件

References

- Kubernetes
  - Dynamic Resource Allocation
- Kind, Slurm
- Slinky Project
- AWS Parallel Cluster
- Prometheus Slurm Exporter
- Running Slurm on Amazon EKS with Slinky
- Lmod, Spack
- TBD